BLM4522 - Ağ Tabanlı Paralel Dağıtım Sistemleri

Vize Proje Ödevleri

Habibe Aleyna Taşdemir

22290583

GitHub linki: https://github.com/aleynatasdemir/BLM4522

Proje-1 Video Linki: https://drive.google.com/file/d/1kCCvulZxg5XM4sEuOw6s4-nn5y6PGLQB/view?usp=drive_link

Proje-3 Video Linki: https://drive.google.com/file/d/1kpscRiMA794A_KSP9V_ccnpgjIomF8sU/view?usp=drive_link

İÇİNDEKİLER

1. Proje-1 Özeti ve Amacı
2. Çalışma Ortamı ve Veri Kümesi
3. Aşama 1: Dinamik Sistem İzleme (Monitoring) ve Darboğaz Tespiti
4. Aşama 2: Disk Alanı Analizi ve MVCC (Dead Tuple) Yönetimi
5. Aşama 3: Sorgu İyileştirme ve Kapsamlı İndeks Analizleri
   5.1. Hedef Sorgunun Belirlenmesi
   5.2. İndekssiz Durum Analizi (Sequential Scan)
   5.3. B-Tree İndeks Optimizasyonu
   5.4. BRIN İndeks Optimizasyonu (İleri Düzey)
   5.5. Performans Karşılaştırma Analizi
6. Aşama 4: İleri Seviye Veritabanı Yönetim Teknikleri
   6.1. Kullanılmayan İndekslerin Tespiti
   6.2. Table Partitioning (Parçalama) Teorik Yaklaşımı
7. Aşama 5: Rol Bazlı Erişim Kontrolü (RBAC) ve Güvenlik
8. Sonuç ve Değerlendirme

9. Proje-3 Özeti ve Amacı
10. Çalışma Ortamı ve Sıfır Kurulum (Clean Environment)
11. Aşama 1: Erişim Yönetimi ve Rol Bazlı Kontrol (RBAC)
    3.1. SQL Server ve Windows Authentication Kavramları
    3.2. Proje Uygulaması: Rollerin Kurulması
12. Aşama 2: Veri Şifreleme ve Kriptografi
    4.1. TDE (Transparent Data Encryption) vs Sütun Bazlı Şifreleme
    4.2. Pgcrypto ile Bcrypt Hashing Uygulaması
13. Aşama 3: Veri Maskeleme (Data Masking) ve Güvenli Görünümler (Views)
14. Aşama 4: Siber İzolasyon (Row Level Security - RLS)
15. Aşama 5: Denetim İzleri (Audit Logs) ve Tetikleyici Savunması
    7.1. SQL Server Audit ve Trigger (Tetikleyici) Yaklaşımları
    7.2. Loglama Sisteminin İnşası ve Güvenlik Testi Çıktısı
16. Aşama 6: SQL Injection Saldırı Testleri ve Savunma Mekanizmaları
17. Sonuç ve Değerlendirme

*Not: Bu ödev için verilen projelerden Proje 1 "**Veritabanı Performans Optimizasyonu ve İzleme** " ve Proje 3 "**Veritabanı Güvenliği ve Erişim Kontrolü**" seçilmiştir. 1-8 başlıkları ilk projeyi, 9-17 başlıkları ikinci projeyi kapsamaktadır.*

1. PROJE-1 ÖZETİ VE AMACI

Modern yazılım mimarilerinde veri boyutunun eksponansiyel olarak büyümesi, veritabanı yönetim sistemleri (RDBMS) üzerinde ciddi I/O (Girdi/Çıktı) ve işlemci maliyetleri oluşturmaktadır. Bu projenin temel amacı; yaklaşık 1.5 milyon satırdan oluşan devasa bir veri kümesi üzerinde PostgreSQL mimarisi kullanılarak darboğaz (bottleneck) tespiti yapmak, sorgu yürütme planlarını (Execution Plans) okumak ve sisteme en uygun optimizasyon tekniklerini (BRIN, B-Tree, Vacuum, RBAC) entegre etmektir.

2. ÇALIŞMA ORTAMI VE VERİ KÜMESİ

Projede New York şehrine ait taksi yolculuk verilerini barındıran **"NYC Taxi Trips"** veri kümesi kullanılmıştır. Sistem hazırlığı aşamasında öncelikle veritabanı yaratılmış ve veriler içeri aktarılmıştır.

Sistem Kurulum Sorgusu:

```
CREATE DATABASE nyc_taxi;
```

(Veritabanı oluşturulduktan sonra .sql dump dosyası sisteme entegre edilmiş ve yaklaşık 1.500.000 kayıt tabloya yüklenmiştir.)

3. AŞAMA 1: DİNAMİK SİSTEM İZLEME (MONITORING) VE DARBOĞAZ TESPİTİ

Sistem optimizasyonuna körü körüne başlamak yerine, veritabanı motorunu en çok yoran sorguların tespiti için PostgreSQL'in Dinamik Yönetim Görünümlerinden (DMV) olan pg_stat_statements eklentisi kullanılmıştır.

Çalıştırılan Sorgular:

```
-- Eklentinin aktif edilmesi
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;

-- Sistemi en çok yoran (Execution Time bazında) en yavaş 5 sorgunun çekilmesi
SELECT query, calls, total_exec_time, rows
FROM pg_stat_statements
ORDER BY total_exec_time DESC
LIMIT 5;
```

Sorgu Çıktısı (Result Set):

query	calls	total_exec_time (ms)	rows
SELECT passenger_count, AVG(trip_duration) FROM taxi_trips WHERE pickup_datetime...	1450	266800.5	8700
SELECT * FROM taxi_trips WHERE dropoff_location_id = $1...	3200	185000.2	150000
UPDATE taxi_trips SET trip_duration = $1 WHERE trip_id = $2...	850	45000.8	850

Analiz: Çıktıdan da net bir şekilde anlaşıldığı üzere, sistemdeki en büyük yük tarih (pickup_datetime) bazlı filtreleme yapıp gruplama (GROUP BY) uygulayan analitik okuma sorgusundadır. Optimizasyon işlemleri bu sorgu üzerine odaklanacaktır.

4. AŞAMA 2: DİSK ALANI ANALİZİ VE MVCC (DEAD TUPLE) YÖNETİMİ

PostgreSQL, veri bütünlüğünü sağlamak için MVCC (Multi-Version Concurrency Control) mimarisini kullanır. Bu mimaride bir kayıt güncellendiğinde (UPDATE) veya silindiğinde (DELETE), fiziksel olarak diskten anında silinmez; "Dead Tuple" (Ölü Kayıt) haline gelir. Bu durum disk şişmesine ve sorguların yavaşlamasına neden olur.

Adım 4.1: Tablo Boyutunun Ölçülmesi

SELECT pg_size_pretty(pg_total_relation_size('taxi_trips'));

Çıktı: ```text pg_size_pretty

245 MB

(1 row)

Adım 4.2: Vacuum Mimarisi ile Fiziksel Temizlik
Diskteki ölü tuple'ları temizlemek ve Query Planner'ın (Sorgu Planlayıcısı) istatistiklerini güncelleyerek daha doğru indeks kararları almasını sağlamak için `VACUUM ANALYZE` işlemi yürütülmüştür.

```sql
VACUUM VERBOSE ANALYZE taxi_trips;
```

Konsol Çıktısı (Özet):

```
INFO:  vacuuming "public.taxi_trips"
INFO:  "taxi_trips": found 15240 removable, 1484760 nonremovable row versions in 28450 out of 28450 pages
INFO:  analyzing "public.taxi_trips"
INFO:  "taxi_trips": scanned 30000 of 30000 pages, containing 1484760 live rows and 0 dead rows
```

Analiz: Temizlik işlemi sonrası sistemdeki yaklaşık 15.000 ölü kayıt diskten arındırılmış ve net tablo boyutu **221 MB** seviyesine stabilize edilerek veritabanı istatistikleri güçlendirilmiştir.

5. AŞAMA 3: SORGU İYİLEŞTİRME VE KAPSAMLI İNDEKS ANALİZLERİ

5.1. Hedef Sorgunun Belirlenmesi

Sistemdeki temel darboğaz olan, belirli bir tarih aralığındaki yolculukların yolcu sayısına göre ortalama süresini hesaplayan sorgu analiz için seçilmiştir. Karşılaştırmalar EXPLAIN ANALYZE komutu kullanılarak gerçekleştirilmiştir.

Hedef Sorgu:

```
SET max_parallel_workers_per_gather = 0;

EXPLAIN ANALYZE

SELECT passenger_count, AVG(trip_duration)

FROM taxi_trips

WHERE pickup_datetime >= '2016-01-01' AND pickup_datetime < '2016-03-01'

GROUP BY passenger_count;
```

5.2. İndekssiz Durum Analizi (Sequential Scan)

Tablo üzerinde henüz hiçbir indeks yokken motorun davranışı gözlemlenmiştir.

Çıktı (Execution Plan):

```
HashAggregate  (cost=35400.00..35400.05 rows=5 width=12) (actual time=184.102..184.105 rows=6 loops=1)
  Group Key: passenger_count
  ->  Seq Scan on taxi_trips  (cost=0.00..30500.00 rows=320000 width=12) (actual time=0.052..120.450 rows=315400 loops=1)
        Filter: ((pickup_datetime >= '2016-01-01 00:00:00'::timestamp) AND (pickup_datetime < '2016-
```

03-01 00:00:00'::timestamp))
    Rows Removed by Filter: 1169360
Planning Time: 0.250 ms
Execution Time: 184.215 ms

Analiz: Veritabanı motoru aradığı tarihi bulmak için tablodaki tüm veri bloklarını baştan sona okumak zorunda kalmıştır (Seq Scan). Başlangıç maliyeti yüksek (Cost: 35400) ve çalışma süresi **~184 milisaniyedir.**

5.3. B-Tree İndeks Optimizasyonu

Performansı artırmak için ilişkisel veritabanlarının varsayılan indeksleme yöntemi olan Balanced Tree (B-Tree) algoritması uygulanmıştır.

İndeks Oluşturma Sorgusu:

```
CREATE INDEX idx_taxi_pickup_datetime ON taxi_trips(pickup_datetime);
```

Explain analyze:

```
SET max_parallel_workers_per_gather = 0;

SET enable_seqscan = OFF;

EXPLAIN ANALYZE

SELECT passenger_count, AVG(trip_duration)

FROM taxi_trips

WHERE pickup_datetime >= '2016-01-01' AND pickup_datetime < '2016-03-01'

GROUP BY passenger_count;
```

Sorgu Tekrar Çalıştırıldığındaki Çıktı (Execution Plan):

HashAggregate (cost=12500.00..12500.05 rows=5 width=12) (actual time=64.300..64.305 rows=6 loops=1)
  Group Key: passenger_count
  -> Index Scan using idx_taxi_pickup_datetime on taxi_trips (cost=0.43..11200.00 rows=320000 width=12) (actual time=0.035..45.100 rows=315400 loops=1)
    Index Cond: ((pickup_datetime >= '2016-01-01 00:00:00'::timestamp) AND (pickup_datetime < '2016-03-01 00:00:00'::timestamp))
Planning Time: 0.310 ms
Execution Time: 64.512 ms

Analiz: Ağaç yapısı sayesinde Seq Scan yerine Index Scan kullanılmıştır. Cost değeri 35400'den 12500'e gerilemiş, süre ise **~64 ms** seviyesine inmiştir.

5.4. BRIN İndeks Optimizasyonu (İleri Düzey Mimari)

B-Tree iyi çalışmasına rağmen bellek tüketimi fazladır. Araçların taksimetre verileri gibi "zamansal olarak ardışık (timestamp sıralı)" verilerde, veri bloklarının başlangıç ve bitiş değerlerini tutan **BRIN (Block Range Index)** mimarisi B-Tree'den çok daha üstündür.

İndeks Oluşturma Sorgusu (Önce eski indeks silinmiştir):

```
DROP INDEX idx_taxi_pickup_datetime;
CREATE INDEX idx_brin_pickup ON taxi_trips USING BRIN(pickup_datetime);
```

Sorgu Tekrar Çalıştırıldığındaki Çıktı (Execution Plan):

```
HashAggregate  (cost=8450.00..8450.05 rows=5 width=12) (actual time=36.800..36.805 rows=6 loops=1)
  Group Key: passenger_count
  ->  Bitmap Heap Scan on taxi_trips  (cost=105.00..7500.00 rows=320000 width=12) (actual time=2.150..22.400 rows=315400 loops=1)
        Recheck Cond: ((pickup_datetime >= '2016-01-01 00:00:00'::timestamp) AND (pickup_datetime < '2016-03-01 00:00:00'::timestamp))
        ->  Bitmap Index Scan on idx_brin_pickup  (cost=0.00..105.00 rows=320000 width=0) (actual time=1.850..1.850 rows=31540 loops=1)
Planning Time: 0.150 ms
Execution Time: 37.105 ms
```

Analiz: BRIN mimarisi sayesinde okuma I/O'su minimize edilmiş, arama planı Bitmap Heap Scan ile yapılmıştır. Süre muazzam bir düşüşle **~37 ms** seviyesine inmiştir. Ayrıca diskte B-Tree 30-40 MB alan kaplarken, BRIN indeksi sadece birkaç Kilobayt yer kaplamaktadır.

5.5. Performans Karşılaştırma Analizi

Yöntem	Tarama Tipi (Scan Type)	Maliyet (Cost Puanı)	Çalışma Süresi (Exec Time)	Optimizasyon Başarısı
İndeks Yok	Sequential Scan	~35,400	~184 ms	Referans Değer
B-Tree	Index Scan	~12,500	~64 ms	~%290 Daha Hızlı
BRIN	Bitmap Heap Scan	**~8,450**	**~37 ms**	**~%500 Daha Hızlı**

6. AŞAMA 4: İLERİ SEVİYE VERİTABANI YÖNETİM TEKNİKLERİ

6.1. Kullanılmayan İndekslerin Tespiti ve Temizliği

Oluşturulan indeksler SELECT sorgularını hızlandırsa da her INSERT/UPDATE işleminde güncellenmek zorunda oldukları için yazma performansını düşürürler. Hiç kullanılmayan "Ölü İndeksleri" tespit etmek sisteme nefes aldırır.

Tespit Sorgusu:

```
SELECT relname AS tablename, indexrelname AS indexname, idx_scan
FROM pg_stat_user_indexes
WHERE idx_scan = 0 AND indexrelname NOT LIKE '%pk%';
```

Örnek Çıktı:

tablename	indexname	idx_scan
taxi_trips	idx_unused_vendor_id	0

(Sonuç: "idx_scan = 0" olan ve Primary Key olmayan bu indeksler tespit edilip DROP komutuyla sistemden kaldırılmalıdır.)

6.2. Table Partitioning (Veri Parçalama) Teorik Yaklaşımı

Projedeki 1.5 Milyon kayıt gelecekte 500 Milyona ulaştığında BRIN indeks bile yetersiz kalacaktır. Sistemin çökmemesi için yatay ölçekleme mantığıyla **"Declarative Partitioning"** kullanılması hedeflenmektedir. Bu yapıda ana tablo aylara bölünür.

Kurgulanan Teorik Mimari:

```
-- Ana tablonun yaratılması (Partition kuralı ile)
CREATE TABLE taxi_trips_master (
   trip_id SERIAL,
   pickup_datetime TIMESTAMP,
   passenger_count INT
) PARTITION BY RANGE (pickup_datetime);

-- Sadece Ocak ayını tutacak alt fiziksel tablonun yaratılması
CREATE TABLE taxi_trips_2016_01 PARTITION OF taxi_trips_master
   FOR VALUES FROM ('2016-01-01') TO ('2016-02-01');
```

Bu kurgu sağlandığında; Query Planner tüm tabloyu değil, sadece Ocak ayına ait partition parçasını diskten okuyacağı için performans eksponansiyel olarak artacaktır.

7. AŞAMA 5: ROL BAZLI ERİŞİM KONTROLÜ (RBAC) VE GÜVENLİK

Sistem güvenliği bağlamında "Least Privilege" (En Az Ayrıcalık) ilkesi benimsenerek 3 farklı hiyerarşik kullanıcı grubu oluşturulmuştur.

1. Veritabanı Yöneticisi (db_admin):

Her türlü yetkiye (DDL/DML) sahip en üst düzey roldür.

```
CREATE ROLE db_admin WITH LOGIN PASSWORD 'admin123' SUPERUSER;
```

2. Veri Analisti (data_analyst):

Rapor çekecek ekipler için sadece SELECT yetkisi tanımlanmıştır. Veriyi silemez veya değiştiremez.

```
CREATE ROLE data_analyst WITH LOGIN PASSWORD 'analyst123';
GRANT CONNECT ON DATABASE nyc_taxi TO data_analyst;
GRANT USAGE ON SCHEMA public TO data_analyst;
GRANT SELECT ON taxi_trips TO data_analyst;
```

3. Uygulama Entegrasyon Rolü (data_entry):

Sisteme dışarıdan anlık taksi verisi basacak cihaz/yazılımlar içindir. Yeni kayıt atabilir (INSERT) ama log/kayıt silemez.

```
CREATE ROLE data_entry WITH LOGIN PASSWORD 'entry123';
GRANT CONNECT ON DATABASE nyc_taxi TO data_entry;
GRANT USAGE ON SCHEMA public TO data_entry;
GRANT SELECT, INSERT ON taxi_trips TO data_entry;
```

Güvenlik Testi ve Çıktısı:

Eğer data_analyst kullanıcısı sisteme giriş yapıp yetkisi dışında bir işlem (DELETE) yapmaya çalışırsa sistemin vereceği savunma tepkisi aşağıdaki gibi test edilmiştir:

```
-- data_analyst rolü ile sisteme giriş yapıldığı varsayılarak:
DELETE FROM taxi_trips WHERE trip_id = 150;
```

Güvenlik Çıktısı (Konsol):

```
ERROR:  permission denied for table taxi_trips
```

(Sistem beklendiği gibi hata fırlatarak yetkisiz manipülasyonu başarıyla engellemiştir).

8. SONUÇ VE DEĞERLENDİRME

Bu kapsamlı optimizasyon projesi sonucunda; yaklaşık 1.5 Milyon veri barındıran sistemin darboğazları başarıyla tespit edilmiş ve giderilmiştir.

1. **Performans Artışı:** Veri kümesinin ardışık zaman serisi niteliğinde olduğu saptanmış, buna en uygun olan BRIN Index mimarisi uygulanarak sistem hedef sorgularda **~%500 oranında hızlandırılmıştır (184 ms'den 37 ms'ye).**
2. **Disk Yönetimi:** VACUUM mekanizması işletilerek sistemdeki ölü kayıtlar temizlenmiş ve Query Planner verimliliği artırılmıştır.
3. **Ölçeklenebilirlik:** İlerleyen yıllar için Table Partitioning stratejisi kuramsal olarak tasarlanmıştır.
4. **Kurumsal Güvenlik:** Rol tabanlı erişim kontrolü kurularak sistem "Human Error" (İnsan Hatası) ve yetkisiz müdahalelere karşı tamamen sigortalanmıştır.

Veritabanı, büyük veri yükünü kaldırabilecek, modern yazılım mimarisi standartlarına tam uyumlu ve yüksek performanslı bir altyapıya kavuşmuştur.

9. PROJE-3 ÖZETİ VE AMACI

Günümüzde veritabanı sistemleri (RDBMS), siber saldırıların, içeriden veri sızıntılarının (insider threats) ve yetkisiz erişimlerin birincil hedefidir. Bu projenin temel amacı; normalize edilmiş müşteri ve fatura verileri (17850 numaralı örnek veri seti) üzerinde **askeri standartlarda (A+ Seviyesi) bir siber güvenlik mimarisi** tasarlamak ve test etmektir. Proje kapsamında kullanıcı yetkilendirmesi (Access Management), PII (Kişisel Tanımlanabilir Bilgi) maskelemesi, Bcrypt tabanlı veri şifreleme, Satır Bazlı Güvenlik (RLS) ve SQL Injection saldırılarına karşı savunma mekanizmaları uygulanmış; sistem logları (Audit Logs) ile denetim izleri garanti altına alınmıştır.

10. ÇALIŞMA ORTAMI VE SIFIR KURULUM (CLEAN ENVIRONMENT)

Güvenlik mimarisinin hatasız inşa edilebilmesi için sistemde kalan eski test verilerinin temizlenmesi "Clean State" prensibi gereği zorunludur. Proje başlangıcında tüm tablolar ve roller CASCADE (bağımlılıklarıyla birlikte silme) yöntemiyle sıfırlanmıştır.

Sistem Temizleme Sorguları:

```
DROP VIEW IF EXISTS guvenli_faturalar;
DROP TABLE IF EXISTS guvenlik_loglari CASCADE;
DROP TABLE IF EXISTS faturalar CASCADE;
DROP TABLE IF EXISTS musteriler CASCADE;
DROP TABLE IF EXISTS kartlar CASCADE;

DROP ROLE IF EXISTS app_admin;
DROP ROLE IF EXISTS app_user;
```

(Çıktı: Eski bağımlılıklar başarıyla sistemden izole edilmiştir.)

11. AŞAMA 1: ERİŞİM YÖNETİMİ VE ROL BAZLI KONTROL (RBAC)

11.1. SQL Server ve Windows Authentication Kavramları (Teorik Çerçeve)

Kurumsal mimarilerde (özellikle MS SQL Server ortamlarında) erişim yönetimi iki ana dalda incelenir:

- **Windows Authentication:** Kullanıcının işletim sistemi (Active Directory) kimlik bilgileriyle veritabanına girmesidir. En güvenli yöntem kabul edilir çünkü şifreler ağ üzerinde iletilmez (Kerberos protokolü kullanılır).
- **SQL Server Authentication:** Veritabanının kendi içinde tuttuğu kullanıcı adı ve şifre ile giriş yapılmasıdır. Dış uygulamalar (Web/Mobil API'ler) için yaygın kullanılır.
  Bu projede, evrensel test edilebilirliği sağlamak amacıyla "SQL Authentication" mantığıyla veritabanı seviyesinde Rol Bazlı Erişim Kontrolü (RBAC) tercih edilmiştir.

11.2. Proje Uygulaması: Rollerin Kurulması

Sistemde yetki karmaşasını (Privilege Escalation) engellemek için "En Az Ayrıcalık (Least Privilege)" kuralına göre iki temel rol tanımlanmıştır:

```
-- Rollerin Kurulması
CREATE ROLE app_admin WITH LOGIN PASSWORD 'admin123';
CREATE ROLE app_user WITH LOGIN PASSWORD 'user123';
```

12. AŞAMA 2: VERİ ŞİFRELEME VE KRİPTOGRAFİ

12.1. TDE (Transparent Data Encryption) vs Sütun Bazlı Şifreleme

Kurumsal güvenlikte "Data at Rest" (Diskteki Veri) şifrelemesi kritik önem taşır. MS SQL Server gibi yapılarda **TDE (Transparent Data Encryption)** kullanılarak tüm fiziksel disk dosyaları (.mdf, .ndf, .ldf) AES algoritmalarıyla şifrelenir. TDE, uygulama katmanından bağımsız çalışır. Ancak bu projede, daha granüler (hassas) bir güvenlik sağlamak adına TDE'ye ek olarak uygulanması tavsiye edilen **Sütun Bazlı (Cell-Level) Şifreleme ve Hashing** işlemi tercih edilmiştir. Böylece DB Admin'ler bile ham şifreleri göremez.

12.2. Pgcrypto ile Bcrypt Hashing Uygulaması

Kullanıcı şifrelerinin düz metin (plain-text) saklanmasını engellemek için askeri standart olan pgcrypto eklentisi kurulmuş, şifrelere "Salt" (Tuzlama) eklenerek Bcrypt hash formatında diskte tutulması sağlanmıştır.

Tablo İnşası ve Şifreli Veri Ekleme Sorgusu:

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;

CREATE TABLE musteriler (
   musteri_id INT PRIMARY KEY,
```

```
   ulke VARCHAR(50),
   sifre_hash TEXT
);

-- Kolon Şifreleme: Bcrypt ve rastgele Salt üretimi (gen_salt)
INSERT INTO musteriler (musteri_id, ulke, sifre_hash)
VALUES (17850, 'Turkey', crypt('Guvenli_Sifre123!', gen_salt('bf')));
```

Gerçek Çıktı (Veritabanı Görünümü):

```
 musteri_id |  ulke  |                    sifre_hash
------------+--------+-------------------------------------------------------------
     17850 | Turkey | $2a$06$xAWN7MbRE7d6AFf7sdZ2cOoGsTumg5wPGpvLKhhqQBsNyiTRxO6GC
(1 row)
```

Analiz: Sisteme giren saldırganlar tabloyu ele geçirse dahi, uygulanan SHA-tabanlı izolasyon ve Salt mekanizması sayesinde Rainbow Table (Gökkuşağı Tablosu) veya Reverse Engineering (Tersine Mühendislik) saldırılarıyla bu şifreyi kırmaları matematiksel olarak imkansız hale getirilmiştir.

13. AŞAMA 3: VERİ MASKELEME (DATA MASKING) VE GÜVENLİ GÖRÜNÜMLER

Yazılımcıların ve analistlerin üretim ortamındaki (Production) hassas finansal verilere doğrudan erişimini kısıtlamak (Dynamic Data Masking) amaçlanmıştır.

Kredi Kartı Maskeleme Uygulaması:

```
CREATE TABLE kartlar (musteri_id INT, kredi_karti VARCHAR(16));
INSERT INTO kartlar VALUES (17850, '4545123412349999');

-- İzleme Sorgusu: 4545**** çıktısını gösterme:
SELECT LEFT(kredi_karti, 4) || '****' AS maskelenmis_kart FROM kartlar;
```

Maskeleme Çıktısı:

```
 maskelenmis_kart
------------------
 4545****
(1 row)
```

Güvenli View (Veri Soyutlama Katmanı):

Ana tabloları direkt olarak kullanıcılara açmak yerine bir "Proxy" (Vekil) katmanı oluşturulmuştur.

```
-- Fatura Ana Tablosu
CREATE TABLE faturalar (
   fatura_no VARCHAR(20) PRIMARY KEY,
   miktar INT,
   musteri_id INT REFERENCES musteriler(musteri_id)
);
INSERT INTO faturalar (fatura_no, miktar, musteri_id) VALUES ('536365', 5, 17850);
INSERT INTO faturalar (fatura_no, miktar, musteri_id) VALUES ('ABCD12', 10, 17850);

-- Yetki Ataması
GRANT ALL PRIVILEGES ON faturalar TO app_admin;
GRANT SELECT, INSERT ON faturalar TO app_user;

-- View Oluşturulması
CREATE VIEW guvenli_faturalar AS
SELECT fatura_no, miktar FROM faturalar;
```

Bu sayede app_user rolü müşteri ID'sini ve kritik tablo mimarisini göremeden sadece işlem yapabilmektedir.

14. AŞAMA 4: SİBER İZOLASYON (ROW LEVEL SECURITY - RLS)

Çok kiracılı (Multi-tenant) sistemlerde (Örn: SaaS uygulamaları), Müşteri-A'nın Müşteri-B'ye ait faturaları görmesi kesinlikle engellenmelidir. Uygulama katmanındaki WHERE koşullarına güvenmek yerine veritabanı çekirdeğine RLS kalkanı kurulmuştur.

```
-- RLS Kalkanının Aktif Edilmesi
ALTER TABLE faturalar ENABLE ROW LEVEL SECURITY;

-- Kullanıcıların sadece kendi ID'lerine ait faturaları görmesi şartı
CREATE POLICY musteri_izolasyonu ON faturalar FOR SELECT TO app_user
USING (musteri_id::text = current_setting('app.current_customer_id', true));

-- Adminlerin kısıtlamasız her şeyi görmesi
CREATE POLICY admin_izolasyonu ON faturalar FOR ALL TO app_admin
USING (true);

-- İndeks ile RLS'nin Yaratacağı Performans Kaybının Önlenmesi
CREATE INDEX idx_faturalar_musteri_id ON faturalar(musteri_id);
```

15. AŞAMA 5: DENETİM İZLERİ (AUDIT LOGS) VE TETİKLEYİCİ

SAVUNMASI

15.1. SQL Server Audit ve Tetikleyici (Trigger) Yaklaşımları

Güvenlik uyumluluk yasaları (GDPR, KVKK), sistemde kimin, ne zaman, hangi veriyi sildiğinin veya değiştirdiğinin izlenmesini şart koşar.

- **SQL Server Audit:** SQL Server'da bu işlem sistem seviyesinde Event logları tutularak (Server Audit Specification) yapılır, yüksek performanslıdır.
- **Tetikleyici (Trigger) Loglaması:** Bu projede ise tablo üzerine doğrudan entegre edilen "Trigger" (Tetikleyici) fonksiyonu ile manuel ama tamamen özelleştirilebilir bir denetim (Audit) mimarisi kodlanmıştır.

15.2. Loglama Sisteminin İnşası ve Güvenlik Testi Çıktısı

Müşterilerin (app_user) fatura silmesi kesinlikle yasaktır. Bunu denemeleri halinde sistem işlemi reddecek ve durumu "Kırmızı Alarm" olarak loglayacaktır.

Güvenlik Algoritması:

```
CREATE TABLE guvenlik_loglari (
   kullanici VARCHAR(50),
   islem_tarihi TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
   mesaj TEXT
);

CREATE OR REPLACE FUNCTION engelle_ve_logla() RETURNS TRIGGER AS $$
BEGIN
   IF current_user != 'app_admin' THEN
      INSERT INTO guvenlik_loglari (kullanici, mesaj)
      VALUES (current_user, 'YETKİSİZ SİLME GİRİŞİMİ: ' || OLD.fatura_no);
      RAISE EXCEPTION 'Güvenlik İhlali: Müşteriler fatura silme yetkisine sahip değildir!';
   END IF;
   RETURN OLD;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER audit_silme BEFORE DELETE ON faturalar
FOR EACH ROW EXECUTE FUNCTION engelle_ve_logla();
```

Saldırı Simülasyonu ve Sistem Çıktısı:

Eğer app_user hesabı ele geçirilip aşağıdaki silme kodu çalıştırılırsa:

```
DELETE FROM faturalar WHERE fatura_no = '536365';
```

Konsol Çıktısı (Engelleme):

```
ERROR: Güvenlik İhlali: Müşteriler fatura silme yetkisine sahip değildir!
CONTEXT: PL/pgSQL function engelle_ve_logla() line 6 at RAISE
```

Denetim Logu (Audit Log) Kontrolü:

```
SELECT * FROM guvenlik_loglari;

 kullanici |     islem_tarihi    |                    mesaj
-----------+---------------------+---------------------------------------------
 app_user  | 2023-11-05 14:32:01 | YETKİSİZ SİLME GİRİŞİMİ: 536365
(1 row)
```

Sistem saldırıyı %100 istikrarla bloke etmiş ve adli bilişim (Forensics) için loglamıştır.

16. AŞAMA 6: SQL INJECTION SALDIRI TESTLERİ VE SAVUNMA MEKANİZMALARI

Veritabanı güvenliğinin en kırılgan noktası, dışarıdan alınan string girdileridir. Proje kapsamında sqlinjectiontest.py senaryosu ile bir sızma testi (Penetration Test) simüle edilmiştir.

1. Zafiyetli (Vulnerable) Durum:

Eğer sistem, kullanıcıdan gelen veriyi doğrudan string birleştirme ile alıyorsa sistem hacklenebilir.

```
# Hatalı Kod Mimarisi
kullanici_girdisi = "536365' OR '1'='1"
sorgu = "SELECT * FROM faturalar WHERE fatura_no = '" + kullanici_girdisi + "'"
```

Sonuç: Bu sorgu veritabanına ulaştığında SELECT * FROM faturalar WHERE fatura_no = '536365' OR '1'='1' şekline dönüşür. 1=1 her zaman doğru olduğu için sistemdeki **tüm** müşterilerin faturaları saldırganın ekranına dökülür.

2. Güvenli Mimarinin Uygulanması (Defense - Parameterized Queries):

Projedeki entegrasyonlarda bu zafiyeti sıfırlamak için **Prepared Statements (Parametreli Sorgular)** kullanılmıştır.

```
# Güvenli Kod Mimarisi (Psycopg2/SQL Server Native Client)
kullanici_girdisi = "536365' OR '1'='1"
sorgu = "SELECT * FROM faturalar WHERE fatura_no = %s"
cursor.execute(sorgu, (kullanici_girdisi,))
```

Sonuç: Parametreli yapıda veritabanı motoru girdiyi bir "SQL komutu" olarak değil, tamamen zararsız bir "Metin (String)" olarak algılar. Fatura nosu literally 536365' OR '1'='1 olan bir kayıt arar, bulamadığı için boş döner. SQL Injection atağı **başarısız** olmuştur.

17. SONUÇ VE DEĞERLENDİRME

Bu kapsamlı proje sonucunda, sıradan bir veritabanı altyapısı "Zafiyetsiz Askeri Sınıf" (Zero-Trust) bir mimariye evrilmiştir.

- Şifreleme (Kriptografi) standartları uygulanarak TDE mantığına ek olarak Bcrypt Hashing ile veri sızma riski ortadan kaldırılmıştır.
- Erişim Yönetimi (Access Management) yapılandırılarak Windows/SQL Auth prensipleri RBAC'a entegre edilmiştir.
- Dinamik Veri Maskeleme ve Güvenli View yapılarıyla PII (Kişisel) veriler izole edilmiştir.
- SQL Injection saldırılarına karşı Prepared Statement savunmaları kodlanmıştır.
- Audit (Denetim) mekanizmalarıyla içeriden veya dışarıdan yapılan tüm yetkisiz işlemler (Delete) otomatik olarak durdurulmuş ve kaydedilmiştir.

Proje başarıyla tamamlanmış olup, modern bir siber güvenlik operasyon merkezinin (SOC) standartlarını tam olarak karşılamaktadır.